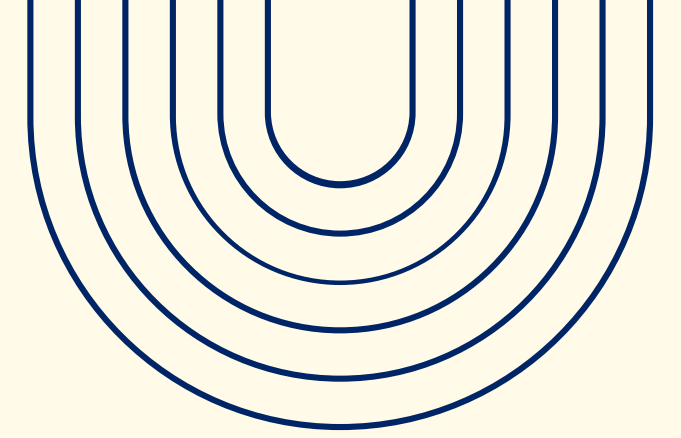
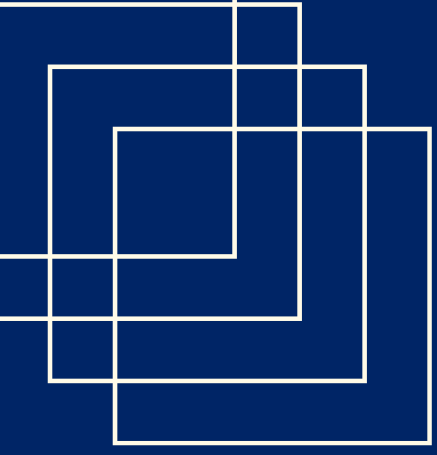


Wearable Stress & Exercise Detection

*Klassifikation
physiologischer Zustände
mit verschiedenen ML
Modellen*



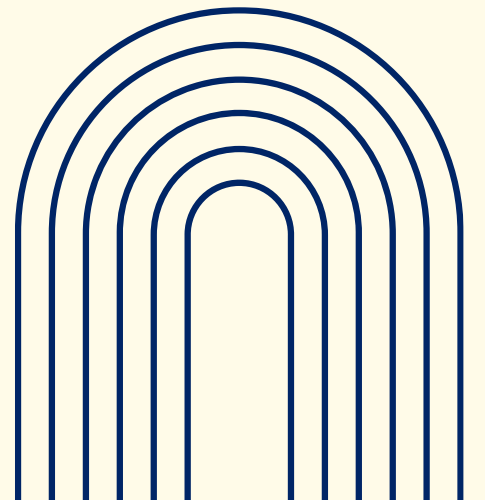
**Welche Eigenschaften
und Besonderheiten
weist der gewählte
Datensatz auf?**

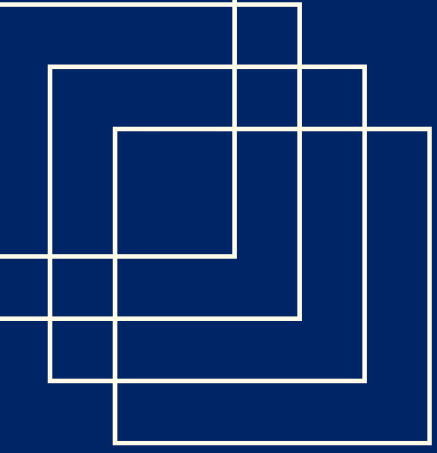


Welche Eigenschaften und Besonderheiten weist der gewählte Datensatz auf?

Er enthält physiologische Messungen von 36 gesunden Freiwilligen, die drei verschiedene Protokolle absolvierten:

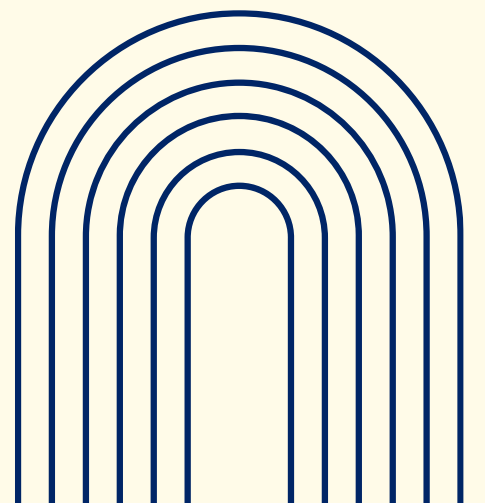
- ein Stress-Induktions-Protokoll,
- ein aerobes Fahrrad-Protokoll und einen (ausdauer)
- anaeroben Wingate-Sprint-Test (sprint)



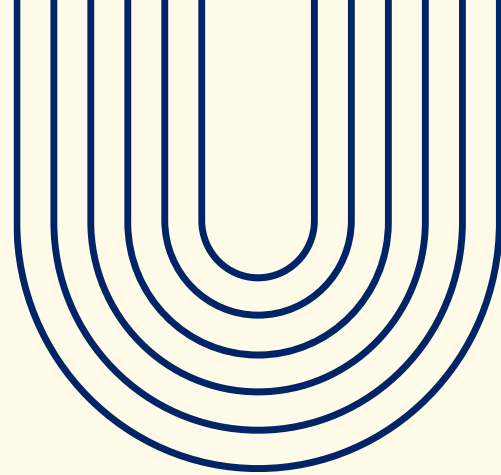


Welche Eigenschaften und Besonderheiten weist der gewählte Datensatz auf?

- Alle Daten wurden mit dem Empatica E4 Wristband aufgezeichnet
- Das Wristband speichert Signale ohne individuelle Zeitstempel pro Messwert
- jede Datei enthält nur einen einzigen Session-Startzeitpunkt (UTC) im Header sowie die Abtastrate



Protokolle



Stress Protokoll

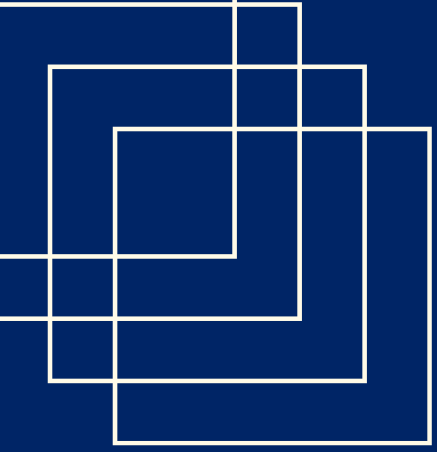
- Probanden saßen in ruhigen Raum mit Kopfhörern
- bekamen verschiedene Aufgaben (wie Mathe, Opinion Tests, Substract Test) und dazwischen Ruhepausen
- mussten vor und nach jeder Aufgabe das subjektive Stresslevel von 1-10 wiedergeben

Anaerobes Protokoll

- dreiminütige Basisphase zum Aufwärmen
- darauf 3 Zyklen mit jeweils 30 Sekunden max Leistug gegen hohen Widerstand gefolgt von dreiminütigen Erholungsphasen ohne Widerstand
- anschließend 2 minütige Ruhepause ohne Bewegung

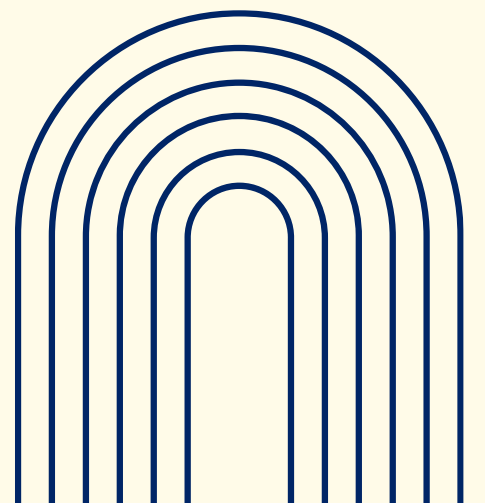
Aerobes Protokoll

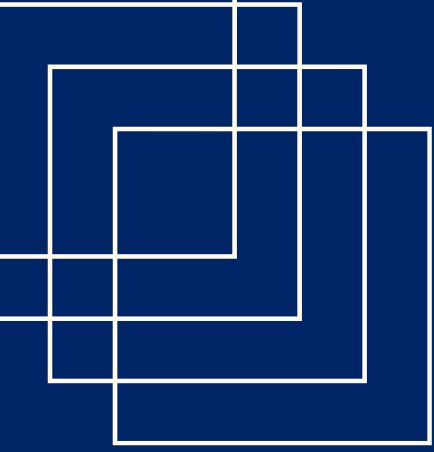
- umfasst etwas 35 min kontinuierliches Radfahren
- Widerstnad wurde Schrittweise erhöht
- Abschließend vier minuten Abkühlphase ohne Widerstand und zwei Minuten Ruhe im Stillstand



Dateiformat

- Jedes Signal liegt als eigene CSV-Datei vor:
 - Zeile 1: Unix-Zeitstempel (UTC) des Sitzungsbeginns
 - Zeile 2: Sample Rate in Hz
 - Ab Zeile 3: die eigentlichen Messwerte (eine Zahl pro Zeile bei EDA/HR/TEMP/BVP, drei Zahlen pro Zeile bei ACC)
- Zusätzlich gibt es eine tags.csv, die die Unix-Zeitstempel der manuell gesetzten Protokollmarker (Knopfdrücke) enthält





Daten pro Teilnehmer

Signal	Abkürzung	Einheit	Sample Rate	Was wird gemessen?
Elektrodermale Aktivität	EDA	μS (Mikrosiemens)	4 Hz	Hautleitfähigkeit – reagiert auf Stressereignisse über das sympathische Nervensystem
Blutvolumenpuls	BVP	normiert (a.u.)	64 Hz	Infrarot-Reflexion an der Fingerarterie – Basis für Herzrate & HRV
Herzrate	HR	bpm	1 Hz	Herzschläge pro Minute, vorverarbeitet vom Gerät
Accelerometer	ACC	g	32 Hz	Bewegung in X-, Y-, Z-Achse (je 1 Spalte)
Hauttemperatur	TEMP	$^{\circ}\text{C}$	4 Hz	Handgelenk-Oberflächentemperatur

Besonderheiten:

Es gibt 2 verschiedene Versionen des Protokolls:

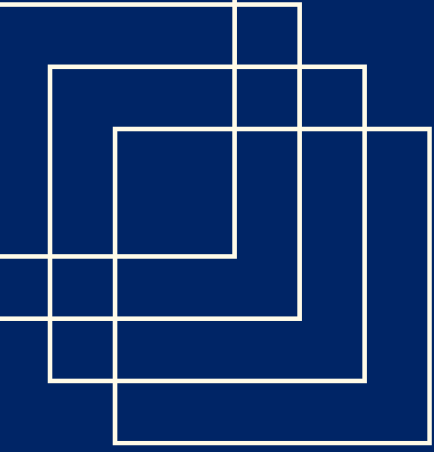
Version 01

Ursprüngliche Version

Version 02

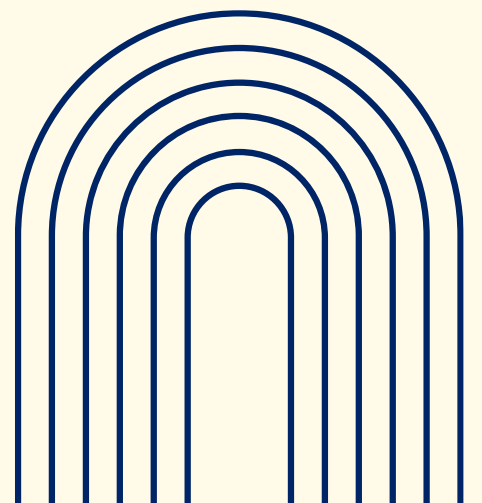
- Stressprotokoll wurden angepasst (Ruhezeiten verlängert und entspanntes Video eingesetzt)
- aerobe Protokoll angepasst (verkürzt)
- aerobes Protokoll angepasst (zusätzlicher Sprint, auf 45sekunden verlängert)

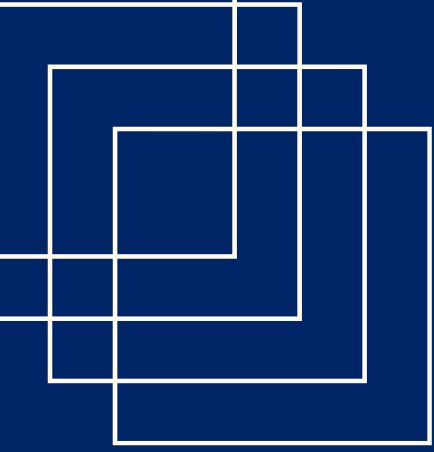




Weitere Besonderheiten

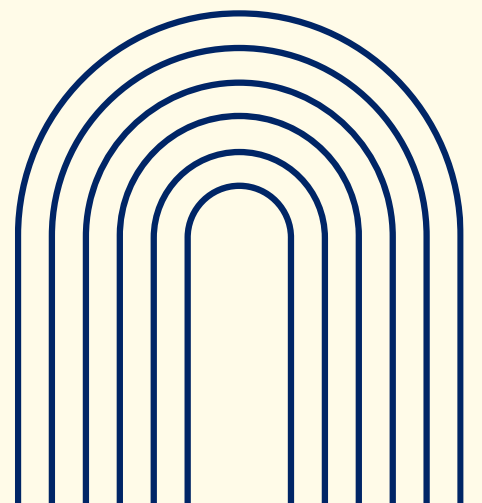
- Die Empatica-Signale haben keine individuellen Zeitstempel pro Messwert
→ nur einen Session-Startzeitpunkt plus die Abtastrate im Datei-Header
- Die Phasen-Markierungen in der Tags-Datei sind aber absolute UTC-Zeitstempel
- Um Signale und Phasen aufeinander abzubilden, rechnen wir jeden Tag in Sekunden seit Session-Start um:
- $\text{offset_sekunden} = \text{tag_UTC} - \text{session_start_UTC}$
- Daraus ergibt sich dann die genaue Sample-Position im Signal:
- $\text{sample_index} = \text{offset_sekunden} \times \text{sample_rate}$

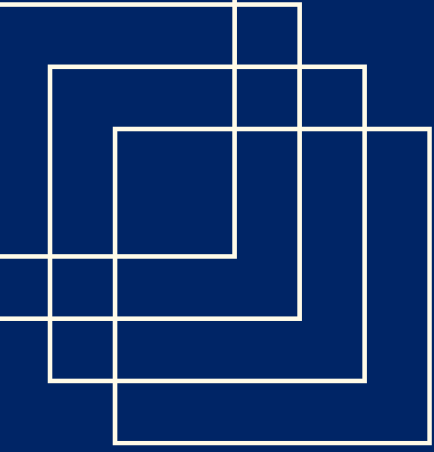




Einschränkungen

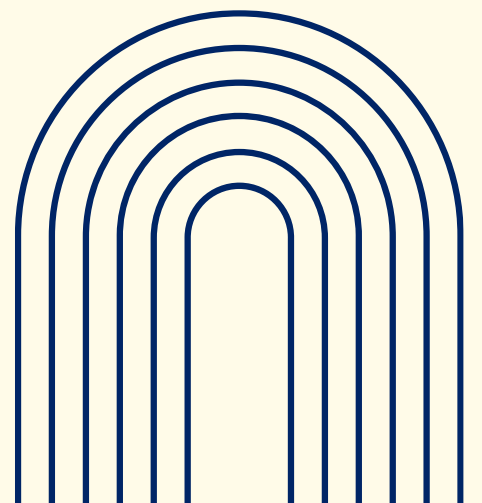
- Klassenungleichgewicht:
- Nach der Segmentierung entstehen deutlich mehr Stress- und Rest-Blöcke als Aerobic- oder Sprint-Blöcke → Das Modell könnte die häufigere Klasse (stress) bevorzugen.
→ Lösung: Oversampling der Minderheitsklasse
- kleine Stichprobe (nur ungefähr 400 Samples) → Deep Learning würde deutlich mehr Daten brauchen
- Ausschließlich junge und gesunde Probanden
- Für ältere Menschen, Übergewichtige oder Diabetiker ist Übertragbarkeit unbekannt
- Tests teils im Freien und zu verschiedenen Tageszeiten durchgeführt → das beeinflusst EDA und Hauttemperatur



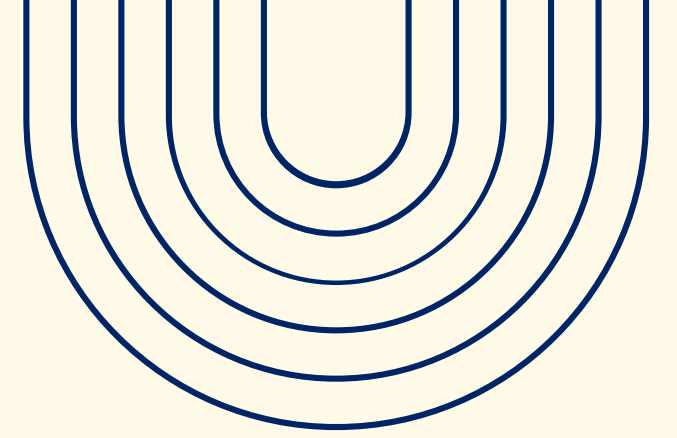


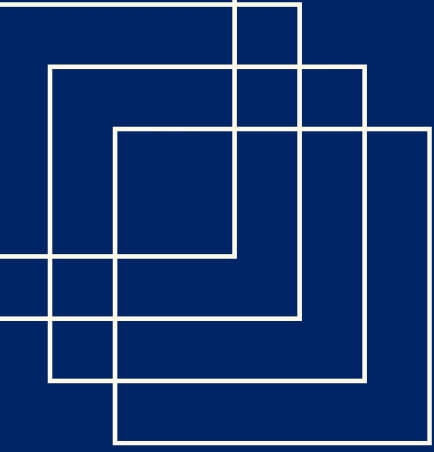
Einschränkungen

- Dataset enthält extra data_constraints Datei
- Datenfehler bei einzelnen Probanden:
 - S02: duplizierte Daten im Stress-Protokoll
 - f07: Armband-Schutzcover nicht entfernt → EDA-Signal ungültig
 - f14: Session in zwei Teile gesplittet (Verbindungsabbruch)
 - S11, S16: Aerobic bzw. Anaerobic ebenfalls gesplittet
 - S12: hat Aerobic-Session gar nicht gemacht
- Stress und Anaerobic am selben Tag gemacht → es ist möglich, dass Stress-Hormone beim Wingate-Test noch nachwirken
- Aerobic war an einem separaten Tag, um Überlappungen zu vermeiden



**Welche ersten
Vorverarbeitungs-
und
Modellierungsschritte
wurden bereits
durchgeführt?**

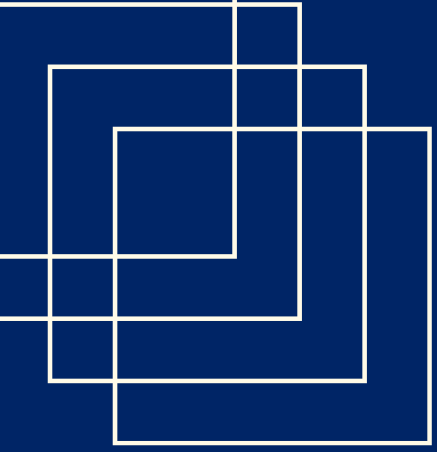




Schritt 1: load_data

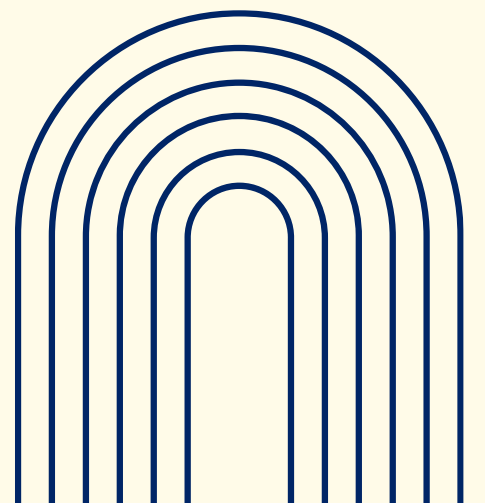
- Funktion, die die fünf Signaldateien des Empatica-Wristbands (Puls, Hautleitfähigkeit, Beschleunigung, Hauttemperatur, Herzrate) einliest
- Das CSV-Format der Daten hat einen speziellen Header: erste Zeile ist der Startzeitpunkt, zweite Zeile die Abtastrate, ab der dritten Zeile kommen die Messwerte

```
def load_signal(file_path):  
    """  
    Liest eine Empatica-CSV-Datei ein.  
    Zeile 1 = Startzeitpunkt (UTC)  
    Zeile 2 = Sample Rate in Hz  
    Ab Zeile 3 = Messwerte  
  
    Gibt zurueck: (daten_array, sample_rate, start_zeit)  
    """  
    # CSV einlesen ohne Header (sonst nimmt pandas die erste Zeile als Spaltennamen)  
    df = pd.read_csv(file_path, header=None)  
  
    # Erste Zeile: Startzeitpunkt  
    start_time = pd.to_datetime(df.iloc[0, 0])  
  
    # Zweite Zeile: Sample Rate  
    sample_rate = float(df.iloc[1, 0])  
  
    # Ab Zeile 3: die echten Daten (als Zahlen)  
    data = df.iloc[2:].reset_index(drop=True).astype(float).values  
  
    return data, sample_rate, start_time
```

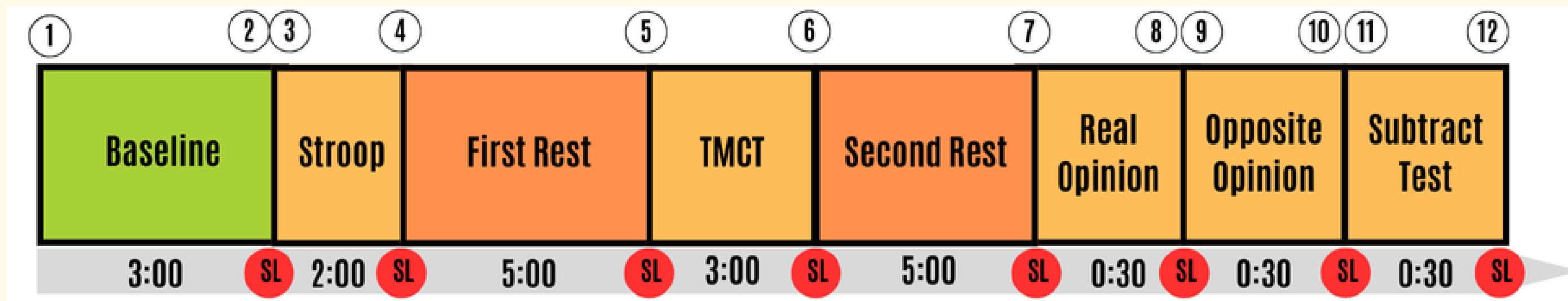
Schritt 1: load_data

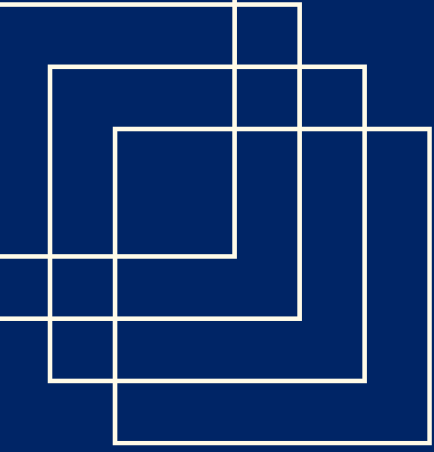
- → Ausgabe: Numpy-Array mit Messwerten + Abtastrate + Startzeitpunkt
- Zusätzlich lesen wir die Marker-Datei ein und rechnen die UTC-Zeitstempel der Knopfdrücke in Sekunden seit Aufnahmebeginn um
- → Ergebnis: Wir können beliebige Probanden-Sessions einheitlich laden



Schritt 2: segment_data

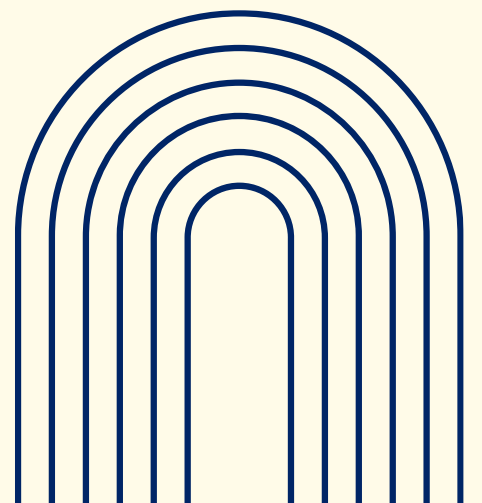
- die physiologischen signale laufen als ein durchgehender Zeitstrahl über die gesamte Sitzung
- → Zeitstrahl muss in klar benannte Abschnitte (Blöcke) zerschnitten werden
- Jeder Block bekommt ein Label, das seinen physiologischen Zustand beschreibt (z.B. Ruhe, Stress, Aerobic..)
- Aus den Knopfdruck-Zeitpunkten rekonstruieren wir, welcher Zeitabschnitt zu welcher Phase gehört

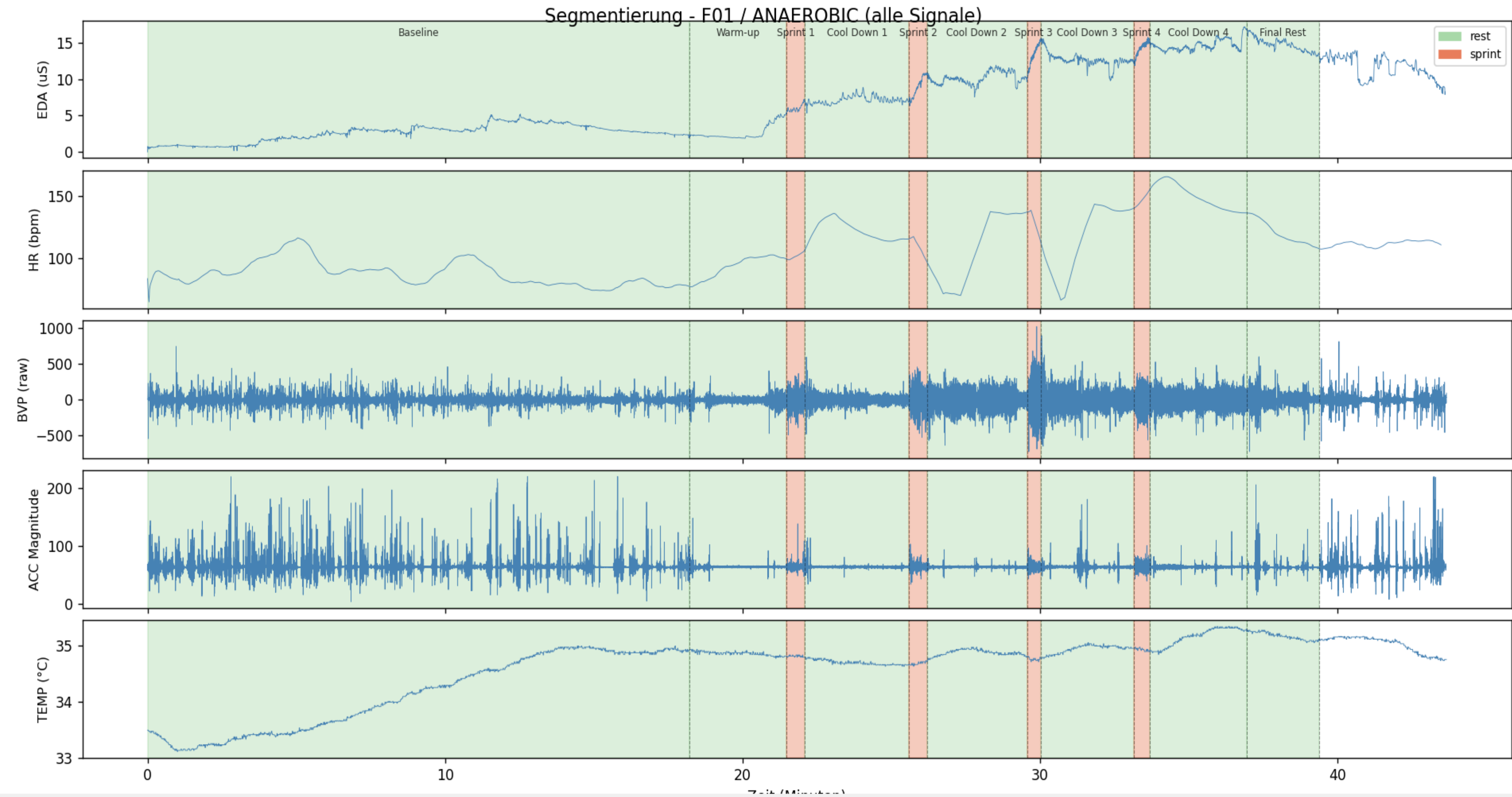


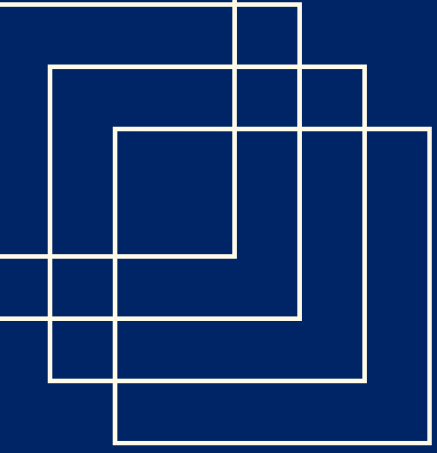


Schritt 2: segment_data

- Besonderheit: Studie nutzt zwei verschiedene Protokollversionen
- → Wir haben sechs Zuordnungsfunktionen implementiert, die richtige wird dann automatisch über das Probanden-Prefix gewählt
- Jeder Block bekommt:
 - einen Phasennamen (z.B. Real Opinion, Subtract Text...)
 - ein Label (rest, stress, aerobic, anaerobic)
 - ein Skip Label für kurze Selbsteinschätzungs Phasen, die wir in der Analyse ignorieren
- `segment_stress_v2()` → ordnet die 9 Button-Presses (Tags) den Protokollphasen zu und gibt für jeden Block (start, end, label, name) zurück

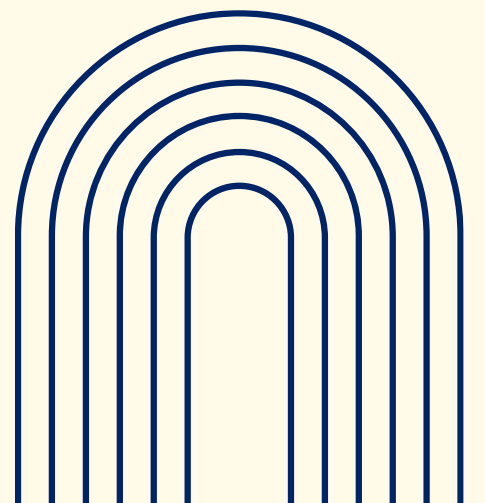


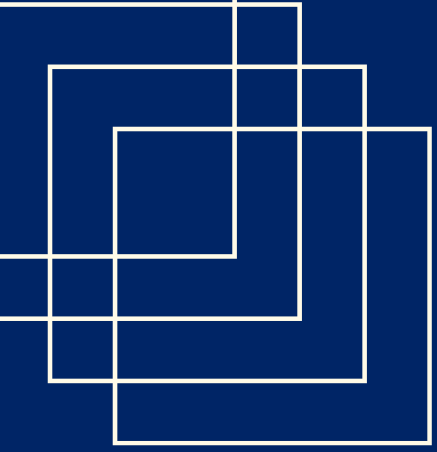




Schritt 3: preprocess

- Die Rohsignale enthalten Rauschen und Bewegungsartefakte
- → Vor der Feature-Berechnung müssen sie bereinigt werden
- Wichtig:
- wir filtern erst das gesamte Signal und schneiden danach erst in Blöcke



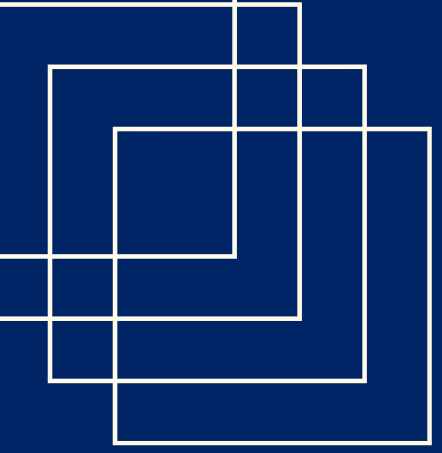


Schritt 3: preprocess

Puls (BVP):

- Bandpass-Filter, der nur den Frequenzbereich echter Herzschläge durchlässt (Chebyshev Typ II Bandpass 0,5 Hz-10Hz)
- → Ausgabe: gefiltertes BVP, aus dem NeuroKit2 zuverlässig Peaks finden kann

```
def filter_bvp(bvp_raw, sample_rate=64):  
    sos = sps.cheby2(4, 40, [0.5, 10], btype="bandpass", fs=sample_rate, output="sos")  
    return sps.sosfiltfilt(sos, bvp_raw)
```

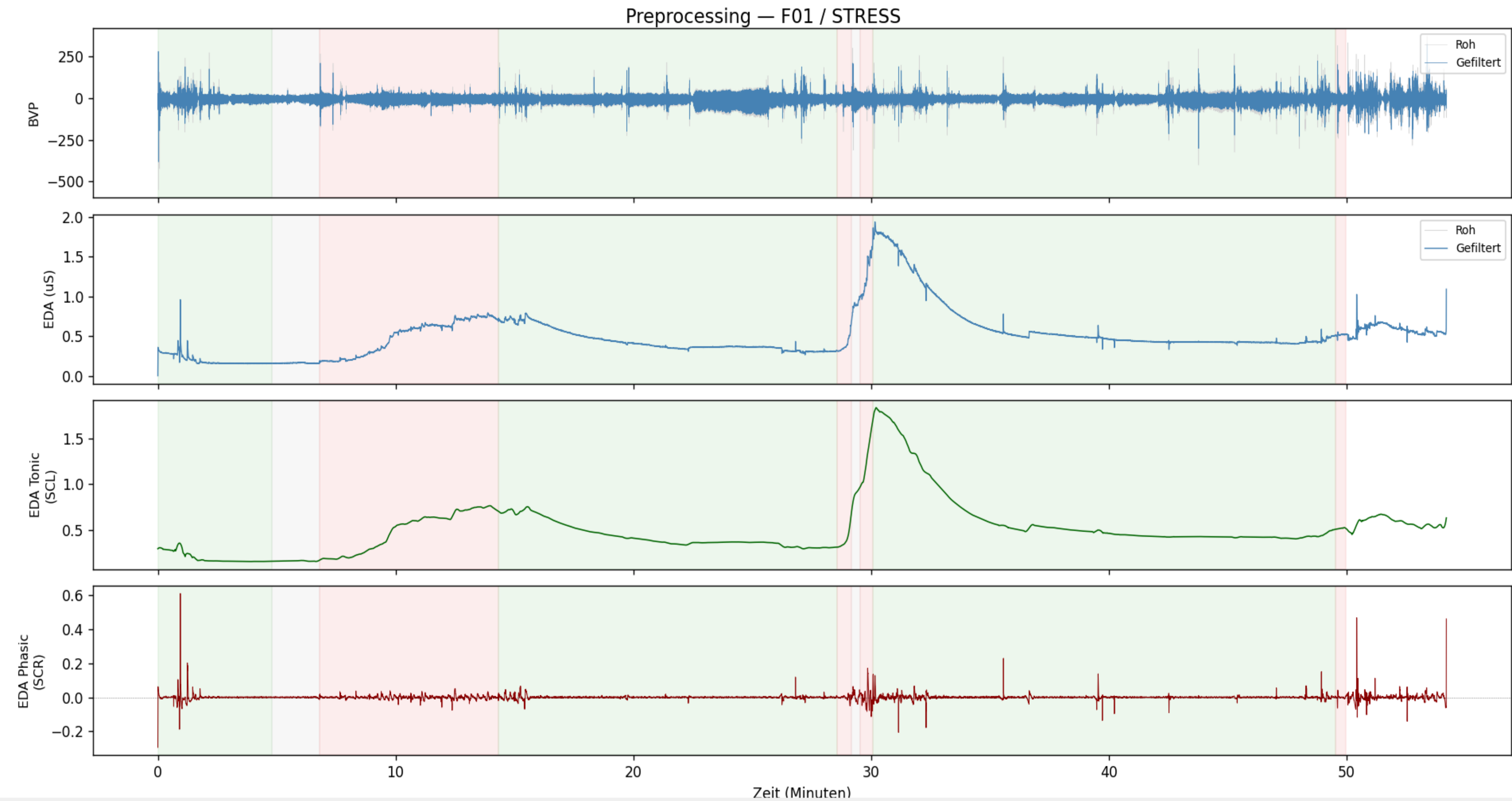


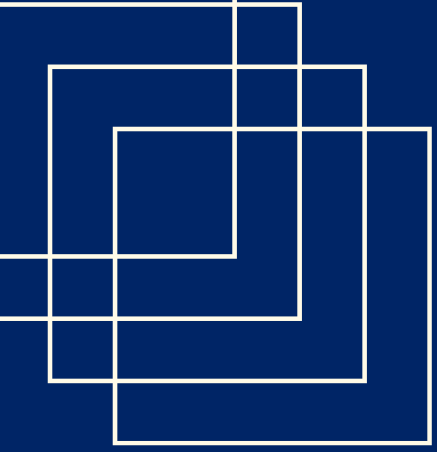
Schritt 3: preprocess

Hautleitfähigkeit (EDA):

- Erst Tiefpass gegen Rauschen (Butterworth Lowpass 1,99Hz)
- dann Glättungsfilter (Savitzky-Golay Filter), der das Signal in zwei Komponenten zerlegt:
 - eine langsame Baseline (Tonic) ("wie gestresst bin ich generell?")
 - schnelle Spikes Phasic ("habe ich gerade auf etwas reagiert?")

```
def filter_eda(eda_raw, sample_rate=4):  
  
    sos = sps.butter(5, 1.99, btype="low", fs=sample_rate, output="sos")  
    eda_filtered = sps.sosfiltfilt(sos, eda_raw)  
  
    window_length = 101 # ~25 Sekunden bei 4 Hz, muss ungerade sein  
    polyorder = 3  
    eda_tonic = sps.savgol_filter(eda_filtered, window_length, polyorder)  
    eda_phasic = eda_filtered - eda_tonic  
  
    return eda_filtered, eda_tonic, eda_phasic
```





Schritt 3: preprocess

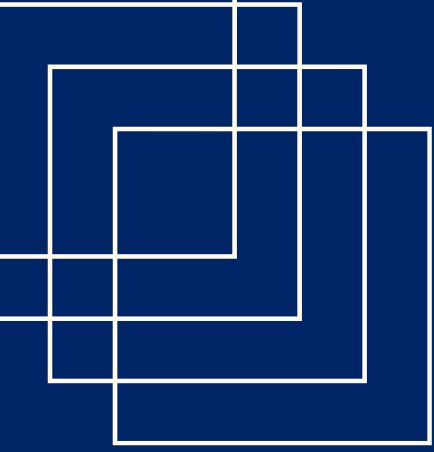
Puls (HR) und Hauttemperatur (TEMP):

- direkt verwendet, kein Filter nötig

Beschleunigung (ACC):

- wir berechnen aus den drei Achsen die Bewegungsintensität → die Orientierung des Wristbands ist dann egal

→ Die Filterparameter haben wir exakt aus dem Paper übernommen

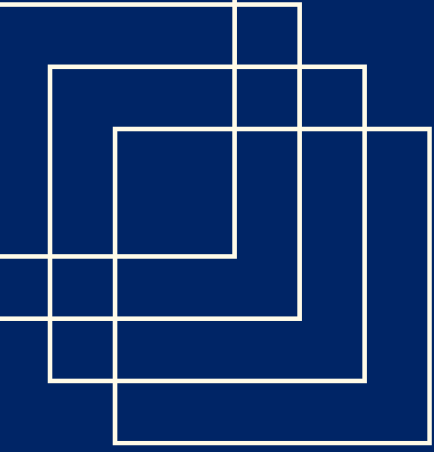


Schritt 4: extract_features

- Für jeden Phasenblock berechnen wir 22 charakteristische Kennzahlen (Features), die den physiologischen Zustand dieses Abschnitts beschreiben

Gruppe	Feature	Berechnung	Physiologische Bedeutung
HR (2)	hr_mean	Mittelwert der Herzrate im Block	Durchschnittliche kardiovaskuläre Aktivierung
HR	hr_std	Standardabweichung der Herzrate	Variabilität / Irregularität der Herzrate
EDA (6)	mean_tonic_eda	Mittelwert von eda_tonic	Allgemeines Erregungsniveau (langsame Baseline)
EDA	std_tonic_eda	Standardabweichung von eda_tonic	Schwankung der Baseline über den Block
EDA	std_phasic_eda	Standardabweichung von eda_phasic	Stärke der einzelnen Stress-Spikes
EDA	tonic_ratio_down	5. Perzentil der 1. Ableitung von eda_tonic	Maximale Abfallrate vom Erregungsniveau
EDA	peaks_density	Anzahl SCR-Peaks pro Minute (via NeuroKit2)	Häufigkeit Stress-Spikes pro Minute
EDA	mean_recoverytime	Mittlere Erholungszeit der SCR-Peaks	Wie schnell normalisiert sich das EDA nach einem Spike

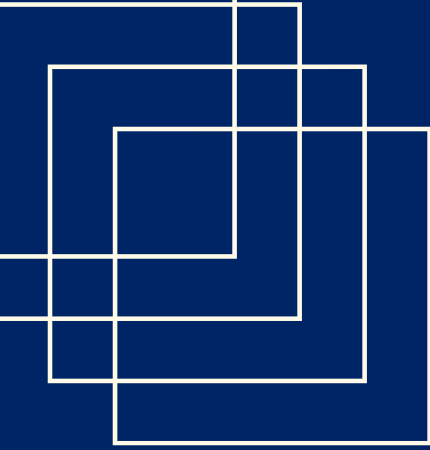
Gruppe	Feature	Berechnung	Physiologische Bedeutung
HRV (6)	max_ibi	Maximales Inter-Beat-Interval (ms)	Längste Pause zwischen zwei Herzschlägen
HRV	min_ibi	Minimales IBI (ms)	Kürzeste Pause – Maximalfrequenz
HRV	ibi_mean	Mittleres IBI (ms)	Durchschnittlicher Herzschlagabstand
HRV	rmssd	Wurzel des mittleren quadr. IBI-Unterschieds	Unregelmäßigkeit des Herzschlags → hoch = entspannt, niedrig = gestresst
HRV	LF_peak	LF-Frequenzband-Peak (~0,1 Hz) via FFT	Dominante Schwingungsfrequenz im Stress Nerv Band (= Low Frequency Band → bestimmter Frequenzbereich in dem Herzrhythmus schwankt, diese langsamen Schwankungen werden vom Sympathikus gesteuert → dem Teil des Nervensystems der bei Stress aktiv wird → gilt als Indikator für Sympathikus Aktivität)
HRV	LF_n	Normalisierte LF-Leistung	Relativer Anteil des LF-Bands am Gesamt-HRV
ACC (5)	x_std / y_std / z_std	Standardabweichung pro Achse	Bewegungsintensität je Raumrichtung
ACC	acc_mean	Mittelwert der Magnitude	Durchschnittliche Gesamtbewegung
ACC	acc_ratio_down	5. Perzentil der Magnitudenableitung	Maximale Verlangsamung der Bewegung
TEMP (3)	temp_mean	Mittelwert der Hauttemperatur	Basistemperatur – Durchblutung
TEMP	temp_std	Standardabweichung der Hauttemperatur	Temperaturvariabilität
TEMP	temp_slope	Steigung der linearen Regression über Zeit	Erwärmt/kühlt sich die Haut im Block?



Schritt 4: extract_features

- Die HRV-Features basieren auf dem gefilterten BVP-Signal
- NeuroKit2 detektiert die Herzschlag-Peaks (PPG-Peaks) mit `ppg_findpeaks()`
- Aus den Abständen zwischen aufeinanderfolgenden Peaks (Inter-Beat-Intervals, IBI in ms) werden dann die Zeitbereichs- und Frequenzbereichsmaße berechnet
- → Bei zu kurzen Blöcken (weniger als 4 Peaks) werden alle HRV-Features als NaN markiert

```
def extract_hrv_features(bvp_block, sample_rate=64):  
    feats = {  
        "max_ibi": np.nan, "min_ibi": np.nan, "ibi_mean": np.nan,  
        "rmssd": np.nan, "LF_peak": np.nan, "LF_n": np.nan,  
    }  
  
    try:  
        # 1) Peaks im BVP finden  
        info = nk.ppg_findpeaks(bvp_block, sampling_rate=sample_rate)  
        peaks = info["PPG_Peaks"]  
        if len(peaks) < 4:  
            return feats
```

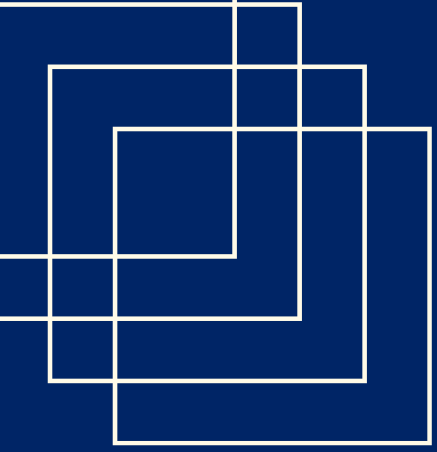


Schritt 4: extract_features

- Für jeden Block (für einen einzigen Probanden und eine Kategorie) entsteht eine Zeile in einem pandas DataFrame mit 28 Spalten (6 Metadaten + 22 Features)

subject	category	phase_name	label	duration_sec	hr_mean	hr_std	mean_tonic_e	std_tonic_e	std_phasic_e	tonic_ratio_d	peaks_density	mean_recover	max_ibi	min_ibi	ibi_mean	rmssd	LF_peak	LF_n
F01	STRESS	Baseline	rest	287.0	71253	2722	0.196	0.059	0.032	-0.002	1045	1062	1718.75	312.5	815089	195747		0.317
F01	STRESS	TMCT	stress	452.0	67851	19054	0.493	0.211	0.01	-0.001	9027	1528	1250.0	328125	860717	188727		0.16
F01	STRESS	First Rest	rest	854.0	66149	23132	0.435	0.13	0.007	-0.001	1827	3588	1453125	328125	900915	191.89		0.168
F01	STRESS	Real Opinion	stress	37.0	72582	1666	0.394	0.103	0.012	0.0	11351	0.4	1078125	453125	747396	142624		0.181
F01	STRESS	Opposite Opinion	stress	32.0	70619	1575	1258	0.229	0.05	0.001	45962	1375	1093.75	312.5	828993	184821		
F01	STRESS	Second Rest	rest	1169.0	64202	4181	0.63	0.364	0.011	-0.002	1129	13938	1515625	312.5	883741	205993		0.177
F01	STRESS	Subtract Test	stress	26.0	73.35	1734	0.515	0.005	0.012	-0.001	9231	0.917	1140625	531.25	857143	161374		

- man sieht dass bei LF_peak keine Daten
- Wenn NeuroKit2 keinen klaren Peak im LF-Band findet, gibt es die Spalte entweder gar nicht zurück oder befüllt sie mit NaN → dann bleibt unser Feature NaN (wahrscheinlich für unseren Datensatz unbrauchbar)



Schritt 4: extract_features

- Wir haben die Pipeline zunächst nur für einen Probanden (f01) implementiert, um jeden einzelnen Schritt → Segmentierung, Filterung, Feature Extraction außer validieren zu können
- Ein Fehler in einem frühen Schritt würde sich sonst durch die ganze Pipeline ziehen und alle Folgeberechnungen verfälschen
- Als nächstes haben wir die Features für alle Probanden berechnet und sie in einer großen Tabelle gespeichert

**Vielen Dank für Ihre
Aufmerksamkeit!**